

UNIVERSIDAD DE LAS FUERZAS ARMADAS – ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE INGENIERÍA EN SOFTWARE

ASIGNATURA: DESARROLLO DE VIDEOJUEGOS

UNIDAD 1

Tres en Raya con Inteligencia Artificial

Proyecto 3: Tic-Tac-Toe (MediumAI, UnbeatableAI y Minimax) – Go (Web) + Angular

---

DOCENTE: ING. CÉSAR VILLACÍS SILVA, MSC.

ESTUDIANTES:

Diana Guerra
Paul Moreno
Germán Cáceres

GRUPO N.º 3

FECHA: NOVIEMBRE 2026

Índice

1. Presentación del proyecto **3**

2. Especificación de Requerimientos (IEEE 830) **3**
2.1. Introducción . . . 3
2.1.1. Propósito . . . 3
2.1.2. Alcance . . . 3
2.1.3. Definiciones, acrónimos y abreviaturas . . . 4
2.1.4. Referencias . . . 4
2.2. Descripción general . . . 4
2.2.1. Perspectiva del producto . . . 4
2.2.2. Funciones del producto . . . 4
2.2.3. Características de los usuarios . . . 5
2.2.4. Restricciones . . . 5
2.2.5. Suposiciones y dependencias . . . 5
2.3. Requerimientos específicos . . . 5
2.3.1. Requerimientos funcionales . . . 5
2.3.2. Requerimientos no funcionales . . . 6

3. Modelo de Inteligencia Artificial **7**
3.1. Variante MediumAI . . . 7
3.2. Variante UnbeatableAI con Minimax . . . 8

4. Diagramas **9**
4.1. Diagrama de casos de uso . . . 9
4.2. Diagrama de secuencia: marcar casilla con Minimax . . . 10
4.3. Diagrama de clases (backend Go) . . . 11

5. Metodología SCRUM + XP **11**
5.1. Roles . . . 12
5.2. Product Backlog . . . 12
5.3. Sprints . . . 13
5.4. Prácticas XP aplicadas . . . 13
5.5. Diseño . . . 13
5.6. Desarrollo . . . 14
5.7. Pruebas . . . 14
5.7.1. Pruebas automatizadas de MediumAI . . . 15
5.7.2. Pruebas automatizadas de UnbeatableAI . . . 15
5.7.3. Pruebas automatizadas de Minimax . . . 16
5.7.4. Integración HTTP, frontend y MediumAI . . . 17
5.7.5. Espacios para evidencias visuales . . . 18

6. Análisis de Resultados: Métricas de Software **21**
6.1. Líneas de código . . . 21
6.2. Otras métricas . . . 23
6.2.1. Entorno y tiempos de pruebas . . . 23
6.2.2. Tiempo de respuesta HTTP . . . 23
6.2.3. Dependencias y compilación frontend . . . 24

6.3. Comportamiento de la variante UnbeatableAI 24

7. Conclusiones y Recomendaciones **25**
7.1. Conclusiones 25
7.2. Recomendaciones 25

1. Presentación del proyecto

Se desarrolló el videojuego *Tres en Raya con Inteligencia Artificial* mediante una arquitectura cliente–servidor. Se utilizó Go para el backend y Angular para el frontend, con comunicación HTTP/JSON a través de una API REST. La lógica del juego y la selección de movimientos se ejecutan en el servidor; la interfaz web permite seleccionar la modalidad, jugar y consultar los resultados.

Tabla 1: Modalidades de inteligencia artificial implementadas

Modalidad	**Tecnología**	**Estado**
MediumAI	Go (Web) + Angular	**Implementado**
UnbeatableAI	Go (Web) + Angular	**Implementado**
UnbeatableAI con Minimax	Go (Web) + Angular	**Implementado**

Se migraron las tres variantes del proyecto C# `Tic-Tac-Toe-Game-Windows-Professor`, desarrollado originalmente con Windows Forms. La migración comprendió la adaptación de tipos, estructuras de datos y controles gráficos a la arquitectura web. En Minimax se conservaron el orden de evaluación, las puntuaciones y los criterios de selección de movimientos. La adaptación del muestreo aleatorio de UnbeatableAI se describe en la sección 6.3. Las versiones de referencia se presentan a continuación:

Tabla 2: Versiones C# utilizadas como referencia

Variante	**Carpeta**	**Clase**
UnbeatableAI	`Tic-Tac-Toe-Versión-IA-Fuerte`	`UnbeatableAIController`
MediumAI	`Tic-Tac-Toe-Versión-IA-Debil`	`MediumAIController`
MinMax	`Tic-Tac-Toe-Version-MiniMax`	`UnbeatableAIController`*

*Misma clase, pero implementada con el algoritmo Minimax en lugar del método numérico.

2. Especificación de Requerimientos (IEEE 830)

2.1. Introducción

2.1.1. Propósito

Se especifican los requisitos funcionales y no funcionales del videojuego web *Tres en Raya con IA*, con base en IEEE 830-1998 [3]. La especificación establece el comportamiento del sistema, sus interfaces y los criterios de verificación de las funcionalidades implementadas.

2.1.2. Alcance

El sistema permite que un usuario juegue al Tres en Raya contra un *Jugador Controlado por la Computadora* (JCC) desde un navegador. La JCC dispone de tres modalidades implementadas: **MediumAI** (1), **UnbeatableAI** (2) y **UnbeatableAI con Minimax** (3). Las dos primeras usan pesos 0/1/3; Minimax usa caracteres en una matriz 3×3. El

sistema comprende la gestión de partidas, la alternancia de turnos, la detección del resultado y su presentación en Angular. La API REST proporciona el estado del tablero y los eventos de cada jugada.

2.1.3. Definiciones, acrónimos y abreviaturas

JCC Jugador Controlado por la Computadora (marca «O»).

Usuario Jugador humano (marca «X»); siempre empieza.

mArrayMoves
Vector V_k de 9 posiciones con los pesos 0/1/3.

API REST Interfaz HTTP/JSON entre el frontend y el backend.

SPA *Single Page Application*, aplicación web de una sola página.

2.1.4. Referencias

- IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications.*
- Villacís, C. et al., *Optimización del Juego del Tres en Raya con Niveles de Dificultad, Utilizando Técnicas Heurísticas de IA*, ESPE.
- Material de clase: *Caso 1*, *Caso 2* y *Caso 3 TTT.*
- Código C# de referencia: `Tic-Tac-Toe-Game-Windows-Professor` (IA fuerte, IA débil y MiniMax).

2.2. Descripción general

2.2.1. Perspectiva del producto

La arquitectura cliente–servidor separa la lógica de juego de la presentación. En el **backend Go** se implementaron las reglas y las decisiones de IA migradas desde C#, accesibles mediante la API REST. En el **frontend Angular** se implementaron la visualización del tablero, el envío de jugadas y la reproducción de los eventos recibidos.

2.2.2. Funciones del producto

Se permite seleccionar la modalidad de IA, iniciar o reiniciar una partida, marcar una casilla libre, recibir la respuesta de la JCC y consultar el resultado, el marcador y el inspector de las modalidades numéricas. MediumAI y UnbeatableAI emiten «¡Ganó X!», «¡Ganó O!» y «¡Empate!» con título «Resultado»; Minimax emite «You Win!», «AI Wins!» e «It's a Draw!» con título «Tic Tac Toe». La API conserva pesos de presentación para Minimax, pero Angular oculta el inspector numérico en ese modo mediante una condición en `app.html`. La visualización de Minimax comprende el tablero y la bitácora de movimientos.

2.2.3. Características de los usuarios

Niños de 7 a 11 años (público objetivo del juego original) y estudiantes. No se requiere conocimiento técnico: basta con hacer clic o usar las teclas 1–9.

2.2.4. Restricciones

- La migración debe conservar el comportamiento funcional de las variantes C#.
- Backend en Go 1.23 o superior, solo con la librería estándar.
- Frontend en Angular 22 con navegadores modernos (Chrome, Edge, Firefox).

2.2.5. Suposiciones y dependencias

El servidor Go está disponible en el puerto 8080; en desarrollo, Angular redirige `/api` mediante `proxy.conf.json`.

2.3. Requerimientos específicos

2.3.1. Requerimientos funcionales

Tabla 3: Requerimientos funcionales

ID	Descripción	Prioridad
RF01	El sistema mostrará un tablero 3×3. La API usa índices 0–8; el teclado de la interfaz admite 1–9.	Alta
RF02	El usuario podrá seleccionar MediumAI (1), UnbeatableAI (2) o Minimax (3), habilitados por el catálogo de la API.	Alta
RF03	El usuario siempre inicia con la marca «X».	Alta
RF04	Una casilla libre recibe X; una ocupada se ignora sin provocar otra jugada de IA. La respuesta expone peso 1 para X. Los índices fuera de 0–8 se rechazan.	Alta
RF05	En UnbeatableAI, en su primera jugada la JCC tomará el centro (4) si está libre; si no, una esquina aleatoria (0, 2, 6, 8). En las jugadas siguientes, si el centro está libre, lo toma.	Alta
RF06	UnbeatableAI aplicará el método numérico: suma 6 ⇒ gana; suma 2 ⇒ bloquea; suma 5 en diagonal ⇒ caso trivial; dos lados adyacentes del usuario ⇒ esquina intermedia (orden 1+3→0, 1+5→2, 3+7→6, 5+7→8); en otro caso, casilla aleatoria.	Alta
RF07	En UnbeatableAI, si la JCC está en el centro y la regla de suma 5 aplica, jugará un lado aleatorio entre 1, 5 y 7 (comportamiento de la versión C# descrito en la sección 6.3).	Alta

ID	Descripción	Prioridad
RF08	Después de cada jugada se comprobarán filas, columnas y diagonales; el tablero lleno sin ganador produce empate. Se conservarán los mensajes de cada variante descritos en Funciones del producto.	Alta
RF09	Al terminar la partida el tablero se deshabilitará (equivalente a `grbTicTacToe.Enabled = false`).	Alta
RF10	El usuario podrá iniciar un juego nuevo en cualquier momento.	Media
RF11	El sistema mostrará un marcador de partidas ganadas, empatadas y perdidas por modo.	Media
RF12	El inspector mostrará pesos 0/1/3 y sumas en las IA numéricas. Minimax expondrá la conversión en arrayMoves solo como dato de presentación; Angular oculta ese panel para el nivel 3.	Baja
RF13	MediumAI tomará primero el centro libre; en otro caso buscará suma 6, luego suma 2 y finalmente una casilla libre aleatoria.	Alta
RF14	Minimax explorará con O maximizador y X minimizador; puntuará $10 - d$, $-10 + d$ o 0, restaurará las casillas y conservará el primer máximo estricto en orden 0–8.	Alta
RF15	Tras un final no habrá más movimientos; un juego nuevo o cambio de nivel reiniciará tablero y turno humano.	Alta

2.3.2. Requerimientos no funcionales

Tabla 4: Requerimientos no funcionales

ID	Tipo	Descripción
RNF01	Fidelidad	Cada modo conservará las reglas de su variante C#. Minimax mantendrá puntuaciones, orden y desempates, sin poda alfa-beta ni nuevas heurísticas. La adaptación del muestreo aleatorio de la IA fuerte se describe en la sección 6.3.
RNF02	Rendimiento	Objetivo local: responder a cada jugada en menos de 50 ms, sin animaciones. El muestreo HTTP descrito en Métricas verifica solo la primera jugada; la aceptación es parcial y no se extrapola a toda carga o despliegue.
RNF03	Usabilidad	Interfaz animada, con sonido opcional y soporte de teclado (1–9 y teclado numérico).
RNF04	Accesibilidad	Respeta `prefers-reduced-motion` y usa etiquetas ARIA en el tablero.

ID	Tipo	Descripción
RNF05	Portabilidad	Funciona en navegadores modernos y es adaptable a celular, tablet y escritorio.
RNF06	Mantenibilidad	Cada modo vive en su propio archivo y cumple una interfaz común (`Controller`).
RNF07	Fiabilidad	UnbeatableAI comprobará disponibilidad antes del muestreo aleatorio restringido. Minimax finalizará su búsqueda finita.
RNF08	Concurrencia	Varias partidas simultáneas, aisladas por sesión, con exclusión mutua.
RNF09	Calidad	Las pruebas de UnbeatableAI y Minimax deberán aprobar con `go test ./...`; `go vet ./...` no emitirá diagnósticos.

3. Modelo de Inteligencia Artificial

En las dos IA numéricas, el tablero se representa como el vector V_k, con $k = 0, \ldots, 8$, en el arreglo `mArrayMoves`. Cada casilla vale 0 (vacía), 1 (usuario) o 3 (JCC). Para cada una de las 8 líneas se calcula la suma de sus tres casillas:

Tabla 5: Interpretación de la suma por línea

Suma	Composición	Acción de la JCC
6	$3 + 3 + 0$	Completar la línea y **ganar** (Caso 2)
2	$1 + 1 + 0$	**Bloquear** al usuario (Caso 1)
5 (diagonal)	$1 + 3 + 1$	Caso trivial: esquina libre o lado aleatorio (Caso 3)

La regla de bloqueo en Go conserva el mismo orden de evaluación que el C#:

```
// pick devuelve el primer casillero vacío de la línea (a, b, d).
func (c *UnbeatableAIController) pick(a, b, d int) int {
       index := 0
       if c.ArrayMoves[a] == 0 { index = a
       } else if c.ArrayMoves[b] == 0 { index = b
       } else if c.ArrayMoves[d] == 0 { index = d }
       c.heuristicSum = c.ArrayMoves[a] + c.ArrayMoves[b] + c.ArrayMoves[d]
       return index
}
```

3.1. Variante MediumAI

Se implementó `MediumAIController` mediante la migración de la variante C# de IA débil, con los pesos originales: 0 para una casilla vacía, 1 para X y 3 para O. El turno humano siempre inicia con X. Después de cada jugada válida, el controlador comprueba las ocho líneas ganadoras y el empate; al terminar, registra el mensaje del resultado y deshabilita el tablero.

La JCC (O) aplica las reglas en este orden:

1. Si el centro (`CenterIndex = 4`) está vacío, lo elige de inmediato.
2. Si el centro está ocupado, recorre filas, columnas y diagonales en el orden de `WinningLines`. Primero busca una suma de 6 $(3+3+0)$ para completar una línea y ganar.
3. Si no hay una suma de 6, busca una suma de 2 $(1+1+0)$ para bloquear la primera amenaza de X, eligiendo la primera casilla vacía de esa línea.
4. Si ninguna regla aplica, elige al azar una casilla vacía.

La prioridad global de suma 6 antes de suma 2 permite que O gane cuando tiene una jugada ganadora disponible, aunque exista una amenaza de X. Esta variante no incluye las reglas de suma 5 ni la estrategia de esquinas y lados de `UnbeatableAI`. El backend expone los pesos por medio de `Weights()` y comunica jugadas y resultados como eventos del tablero; la interfaz Angular los presenta sin decidir movimientos.

3.2. Variante UnbeatableAI con Minimax

Mediante Minimax se selecciona la jugada con el mejor resultado posible frente a un adversario que también busca su mejor resultado. Se migraron los métodos de `UnbeatableAIController.cs` y el flujo de `frmTicTacToe.cs` de la variante `Tic-Tac-Toe-Version-MiniMax`, tomando como referencia su código y la explicación del algoritmo [2]. La implementación se integró como nivel 3 y se habilitó en `controller.go`.

`getBestMove` recorre 0–8 y calcula fila $i/3$ y columna i mód 3. Simula O en cada espacio libre, llama a `minimax(board, 0, false)` y restaura el espacio. Actualiza la mejor jugada solo cuando `score > bestScore`: conserva la primera de igual puntuación. Sin casillas libres devuelve -1.

`minimax` evalúa primero: victoria de O devuelve $10-d$; victoria de X, $-10+d$; tablero lleno sin ganador, 0. En otro caso recorre filas y columnas 0–2: O maximiza y X minimiza, incrementando profundidad y deshaciendo cada movimiento. `evaluateBoard` comprueba filas, columnas y diagonales en ese orden, con valores $+10$, -10 y 0. `isMovesLeft` busca el primer espacio vacío. No se añadieron poda alfa-beta, cachés ni nuevas heurísticas.

Tabla 6: Correspondencia de la variante Minimax C# con Go

Método o estructura C#	Adaptación Go
`char[3,3] board`	`board [3][3]byte`: X, O y espacio
`GetBestMove / Minimax`	`getBestMove / minimax`
`EvaluateBoard / IsMovesLeft`	`evaluateBoard / isMovesLeft`
`ResetBoard`	`InitializeData / InitializeControls`
`OnButtonClick / AIMove`	`PlayUser / PlayNPC`, coordinados por `Form.Click`
`MakeMove / CheckGameEnd`	`makeMove / checkGameEnd`
`CheckWin / IsBoardFull`	`checkWin / isBoardFull`
`MessageBox.Show`	`Board.ShowMessage`: evento con texto y título originales

Go representa los caracteres ASCII con byte y pasa un puntero a la matriz para mantener la simulación y restauración compartidas. Los límites iniciales son los de

`int.MinValue` y `int.MaxValue` de C#. `PlayUser` coloca X, comprueba el final y cede el turno; `Form.Click` despacha `PlayNPC`, que no actúa si la partida terminó. Tras O, se comprueba otra vez el final y se restituye el turno humano. Las validaciones de turno e índice adaptan la API; comprobar ocupación sustituye deshabilitar botones individuales y `Board.Enabled=false` representa el final global.

`Flag` expone el turno; `Weights` convierte vacío a 0, X a 1 y O a 3 para mantener el contrato de presentación. Angular recibe estos datos, aunque oculta el inspector cuando la clave es `unbeatable-minmax`; las sumas no intervienen en Minimax. MediumAI aplica centro, victoria, bloqueo y azar; UnbeatableAI agrega reglas de diagonales, esquinas y lados. Minimax examina las continuaciones legales en lugar de aplicar esas reglas numéricas. La validación se presenta en Pruebas y Métricas.

4. Diagramas

Las figuras 1, 2 y 3 representan casos de uso, comunicación y estructura del sistema implementado.

4.1. Diagrama de casos de uso

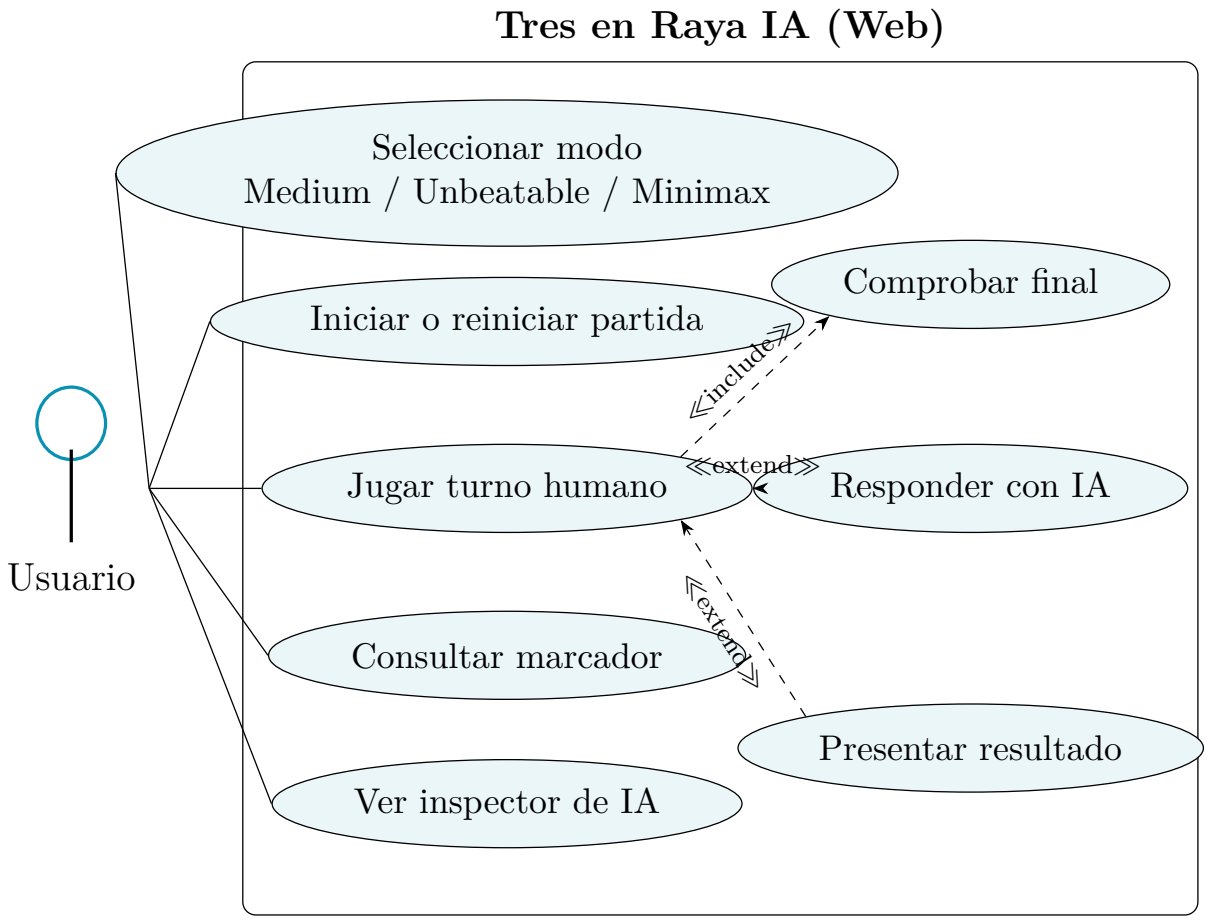


Figura 1: Casos de uso de las tres modalidades. La respuesta IA extiende el turno si la jugada fue aceptada y la partida continúa; el resultado aparece al terminar. El backend es parte del sistema, no un actor externo.

4.2. Diagrama de secuencia: marcar casilla con Minimax

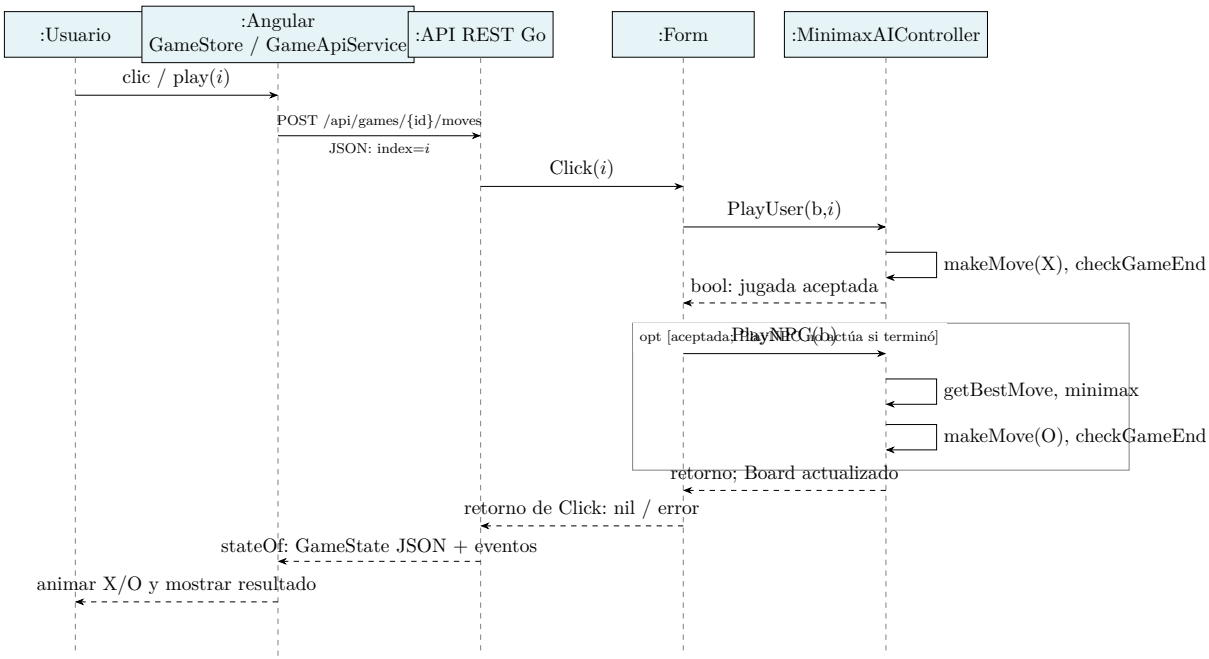


Figura 2: Jugada del nivel 3. La API serializa Board y datos del controlador; Angular reproduce eventos. En niveles 1 y 2 se conserva el despacho por Controller y cambia la estrategia. Una jugada humana terminal no genera O.

4.3. Diagrama de clases (backend Go)

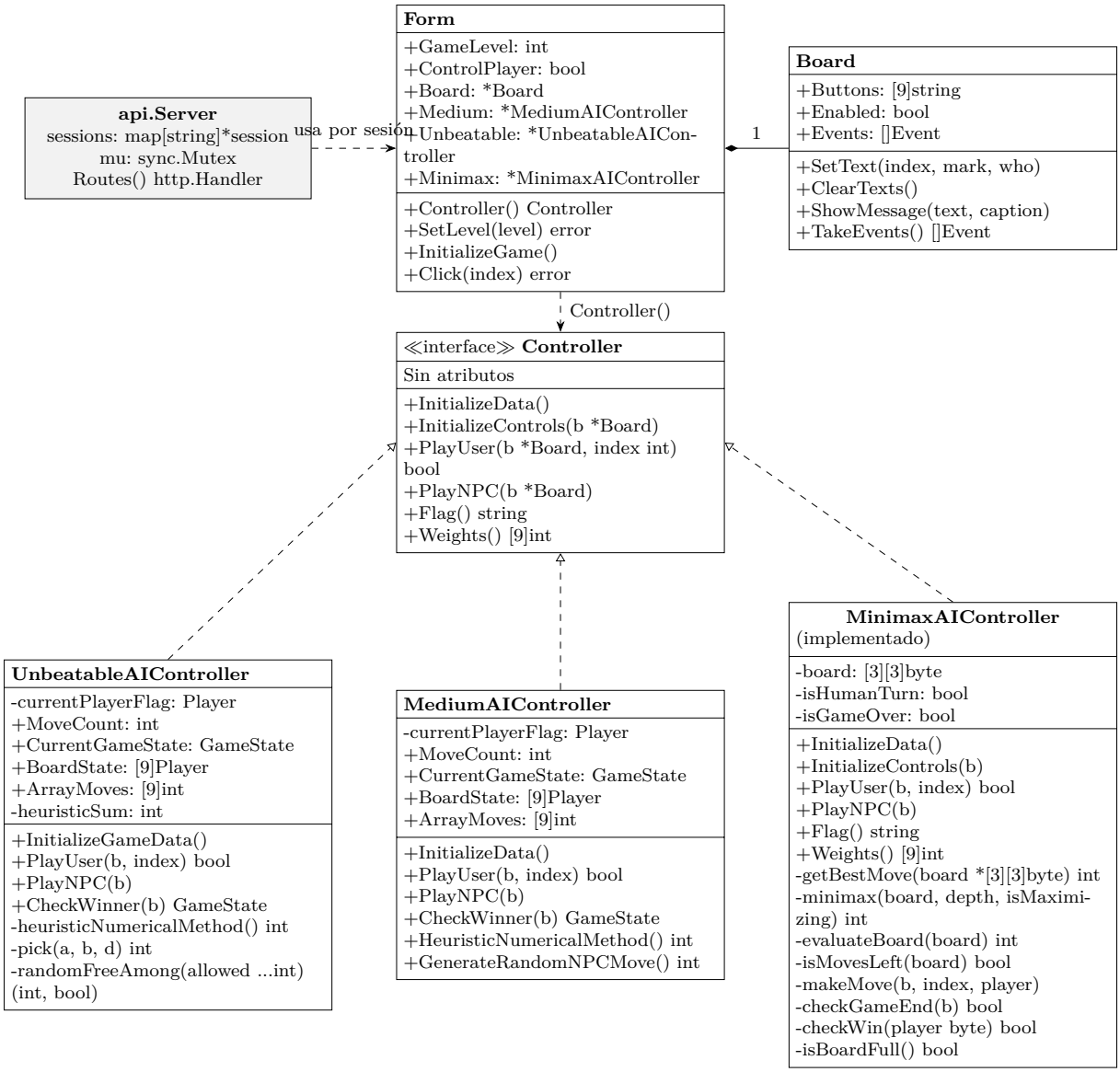


Figura 3: Estructuras e interfaz Go en UML. Los controladores realizan Controller implícitamente; Form conserva una referencia a cada uno y selecciona el activo. El contrato común se abrevia en las IA numéricas. NewForm y NewMinimaxAIController son funciones del paquete, no métodos. No hay herencia de clases en Go.

5. Metodología SCRUM + XP

El desarrollo se organizó en tres sprints, con Scrum como marco de planificación y prácticas XP para el diseño, la implementación y las pruebas. Se distribuyeron las responsabilidades por modalidad de IA y se estructuró el trabajo mediante historias de usuario. La integración se realizó sobre una arquitectura común, con validación automatizada de UnbeatableAI y Minimax.

5.1. Roles

Tabla 7: Distribución de roles y responsabilidades

Rol	**Responsable**
Product Owner	Ing. César Villacís Silva, MSc.
Scrum Master	Diana Guerra
Desarrollo – UnbeatableAI (Go + Angular)	Germán Cáceres
Desarrollo – MediumAI	Paul Moreno
Desarrollo – UnbeatableAI → MinMax	Diana Guerra

5.2. Product Backlog

Tabla 8: Historias de usuario

HU	**Historia**	**Pts**	**Sprint**
HU01	Como usuario, quiero ver un tablero 3×3, para jugar una partida de Tres en Raya.	3	1
HU02	Como usuario, quiero marcar una casilla y recibir la respuesta de la JCC, para alternar los turnos de la partida.	5	1
HU03	Como usuario, quiero enfrentar a UnbeatableAI con reglas de victoria y bloqueo, para jugar contra una estrategia que aproveche oportunidades y responda a mis amenazas.	8	1
HU04	Como usuario, quiero conocer el ganador o el empate, para identificar el resultado de la partida.	3	1
HU05	Como usuario, quiero elegir la modalidad de IA, para jugar contra la estrategia seleccionada.	3	2
HU06	Como usuario, quiero animaciones y sonidos durante la partida, para recibir retroalimentación visual y auditiva de las acciones del juego.	5	2
HU07	Como estudiante, quiero ver los pesos y sumas de las IA numéricas, para comprender sus decisiones.	3	2
HU08	Como usuario, quiero consultar un marcador de resultados por modalidad, para conocer mis partidas ganadas, empatadas y perdidas.	2	2
HU09	Como usuario, quiero jugar contra MediumAI, para enfrentar su estrategia de centro, victoria, bloqueo y selección aleatoria.	5	3
HU10	Como usuario, quiero jugar contra Minimax con las decisiones de la variante C#, para enfrentar una búsqueda completa sin derrotas en la exploración legal.	8	3

5.3. Sprints

Sprint 1 – Núcleo del juego. Migración del método numérico a Go (`unbeatable_ai.go`), tablero (`board.go`), despacho de clics (`form.go`), API REST y pruebas de los Casos 1–3.

Sprint 2 – Experiencia web. Frontend Angular 22: tablero 3D con trazos animados, fondo de partículas, sonidos sintetizados, inspector de IA, marcador y pantalla de resultado con confeti.

Sprint 3 – MediumAI y Minimax. Migración de MediumAI e integración con Controller; finalización de HU10: lectura de originales, traducción de Minimax y del flujo de formulario, eventos de Board, integración, pruebas automatizadas, comparación C#/Go y habilitación del nivel 3.

Se completó HU10 con 2 097 decisiones coincidentes y 569 finales sin derrotas. La validación visual de la interfaz y la incorporación de capturas se mantienen como actividades pendientes de presentación de resultados.

5.4. Prácticas XP aplicadas

- **Diseño simple:** se utilizó un controlador por modalidad y una interfaz común, con reutilización del backend y de las rutas REST.
- **Pruebas automatizadas:** se implementaron pruebas de MediumAI, UnbeatableAI y Minimax para verificar reglas, estados terminales y selección de movimientos.
- **Integración incremental:** se incorporó el nivel 3 sobre Form y Board, manteniendo la separación entre la estrategia de IA y la comunicación web.
- **Estándares de código:** se utilizó gofmt en Minimax y se ejecutó go vet para el análisis estático de Go. Se incluyó Prettier entre las herramientas de desarrollo de Angular.

5.5. Diseño

La arquitectura separa responsabilidades en dos capas:

- **Backend (Go):** `internal/game` contiene la lógica (sin dependencias web) e `internal/api` expone las rutas. `Board` reemplaza a los controles de Windows Forms (`ListButtons`, `GroupBox` y `MessageBox`), de modo que se conservan las reglas originales adaptando estructuras y controles.
- **Frontend (Angular):** componentes *standalone* con *signals*, sin `zone.js`. `GameStore` reproduce los eventos del backend con pausas para animarlos y no decide ninguna jugada.

Tabla 9: API REST

Método	Ruta	Equivale a (original)
GET	/api/modes	Elegir qué aplicación (variante) abrir
POST	/api/games	Abrir el formulario (`frmTicTacToe_Load`)
POST	/api/games/{id}/moves	`btnTicTacToeN_Click`
POST	/api/games/{id}/new	`btnNew_Click`
PUT	/api/games/{id}/level	Cambiar de variante

5.6. Desarrollo

La migración de C# a Go estableció las siguientes correspondencias:

Tabla 10: Correspondencia C# → Go

C# (original)	Go
`ListButtons[i].Text`	`Board.Buttons[i]`
`grbTicTacToe.Enabled`	`Board.Enabled`
`MessageBox.Show(...)`	`Board.ShowMessage(...)` → evento
`enum Player` / `enum GameState`	`type Player int` / `type GameState int`
`mBoardState` (`Player[]`)	`BoardState [9]Player`
`mArrayMoves` (`int[]`)	`ArrayMoves [9]int`
`CENTER_INDEX`, `CORNER_/SIDE_INDICES`	`CenterIndex`, `Corner/SideIndices`
`new Random().Next(0, 9)`	`rand.Intn(9)`
`mHeuristicSum` (miembro)	`heuristicSum` (se conserva entre turnos)
`static bool ControlPlayer`	`Form.ControlPlayer` (por sesión)

En la IA fuerte se incorporó una comprobación de disponibilidad en los `do/while` aleatorios: si no queda ninguna casilla del grupo pedido, el original entra en un bucle infinito. En el servidor esto bloquearía la petición, así que se continúa con la jugada heurística (RNF07).

5.7. Pruebas

Se realizaron pruebas funcionales y de integración el 2 de noviembre de 2026 sobre la versión `5009f27`. Desde el módulo `UNIDAD I/ProjectoUI/Backend` se ejecutaron:

```
go test -count=1 -v ./...
go test -count=1 -coverprofile=<temporal>/game.out ./internal/game
go tool cover -func=<temporal>/game.out
go vet ./...
```

Se aprobaron 24 funciones Test de primer nivel: 8 de MediumAI, 8 de UnbeatableAI y 8 de Minimax. Los subcasos de tablas no se cuentan como funciones adicionales. Go vet finalizó con código 0 y sin diagnósticos. La cobertura de sentencias del paquete internal/game fue de 96,0 % en la ejecución instrumentada.

5.7.1. Pruebas automatizadas de MediumAI

Las pruebas de `medium_ai_test.go` cubren inicialización, turnos, prioridad del centro, las reglas numéricas y los estados terminales.

Tabla 11: Objetivos, escenarios y resultados de MediumAI

ID	Objetivo y procedimiento	Esperado	Obtenido	Estado
D01	Inicializar y reiniciar controlador y tablero.	Pesos vacíos, turno X y controles habilitados.	Estado y tablero restablecidos.	PASS
D02	X marca 0 con el centro libre.	O toma el centro (4).	O en 4; turno X.	PASS
D03	Intentar jugar en el centro ocupado.	Sin cambios en tablero, pesos ni eventos.	Jugada ocupada ignorada.	PASS
D04	Probar las ocho líneas, suma 6/2 y cada casilla vacía.	Ganar o bloquear en el espacio correcto.	48 subcasos aprobados.	PASS
D05	Presentar una victoria y un bloqueo posibles.	Priorizar suma 6.	Selecciona la victoria en 8.	PASS
D06	Sin suma 6/2 y una casilla libre.	Elegir al azar entre libres.	Elige la casilla libre 4.	PASS
D07	Comprobar victoria de X y empate.	Mensaje y tablero deshabilitado.	Ambos finales correctos.	PASS
D08	Explorar 30 recorridos con respuestas aleatorias.	Registrar finales, derrotas y tasa de la IA.	15 982 finales; 590 derrotas (3,69 %).	PASS

5.7.2. Pruebas automatizadas de UnbeatableAI

Las filas siguientes corresponden exclusivamente a `unbeatable_ai_test.go`.

Tabla 12: Objetivos, escenarios y resultados de UnbeatableAI

ID	Objetivo y procedimiento	Esperado	Obtenido	Estado
U01	Primera respuesta a X en 0.	O ocupa el centro.	O en 4.	PASS
U02	Humano ocupa centro; 50 repeticiones.	O elige una esquina.	Las 50 respuestas son esquinas.	PASS
U03	Caso 1: X4, O0; humano marca 2.	Bloqueo en 6.	O en 6.	PASS
U04	Caso 2: O en 2 y 8; humano marca 6.	O gana en 5 y deshabilita tablero.	Jugada, mensaje y final correctos.	PASS

ID	Objetivo y procedimiento	Esperado	Obtenido	Estado
U05	Amenaza y victoria simultáneas; X marca 5.	Priorizar victoria diagonal.	O en 8.	PASS
U06	X en lados 1 y 3; luego 5.	Primera regla de esquina: 0.	O en 0.	PASS
U07	X0, O4; X8; repetir 200 veces.	O en 1, 5 o 7; nunca 3.	Restricción original conservada.	PASS
U08	Recorrer opciones humanas en 30 árboles con IA aleatoria.	Partidas terminan; derrotas de IA no mayores al 3 %.	15 134 finales, 162 derrotas (1,07 %); 0 grupos agotados.	PASS

U01–U08 corresponden, en orden, a `TestFirstMoveTakesCenter`, `TestFirstMoveCornerWhenCenterTaken`, `TestCase1Blocks`, `TestCase2Wins`, `TestTakesImmediateWin`, `TestAdjacentSidesCornerOrder`, `TestSideRuleNeverPicksThree` y `TestOriginalLogicExhaustive`. Este último nombre no implica enumerar todas las decisiones aleatorias: por cada rama humana obtiene una respuesta aleatoria y repite el recorrido 30 veces. Los conteos varían. El resultado citado es de la ejecución general con salida detallada, no de la ejecución posterior instrumentada para cobertura.

5.7.3. Pruebas automatizadas de Minimax

Tabla 13: Objetivos, escenarios y resultados de Minimax

ID	Objetivo y procedimiento	Esperado	Obtenido	Estado
M01	Reiniciar tras victoria de IA; comparar con constructor.	Matriz vacía, turno X y controles habilitados.	Datos y controles reiniciados.	PASS
M02	X0; índices inválidos, doble turno y casillas ocupadas.	Rechazo de ilegales; O4 y pesos 1/3.	Turnos, pesos y eventos correctos.	PASS
M03	Tablero vacío, aperturas, victoria, bloqueo y tablero lleno.	Índices 0, 4, 0, 2, 2, −1; restauración íntegra.	Índices y matriz correctos.	PASS
M04	Ocho líneas con X y O; profundidades 0, 1 y 4.	$+10/-10$ y ajuste original por profundidad.	Líneas, puntuaciones y orden correctos.	PASS
M05	Completar finales e intentar más movimientos.	Mensajes originales y ninguna jugada posterior.	Cuatro escenarios terminales correctos.	PASS

ID	Objetivo y procedimiento	Esperado	Obtenido	Estado
M06	Ramificar todos los movimientos humanos contra IA determinista.	Ninguna victoria humana.	569 finales: 386 victorias IA, 183 empates, 0 derrotas.	PASS
M07	Enumerar estados legales; comparar decisiones con C#.	Mismos conteos y SHA-256.	5 478 estados; 2 097 decisiones coincidentes.	PASS
M08	NewForm(3), Click, InitializeGame y SetLevel.	Despacho, rechazo y reinicio correctos.	Sin O extra tras final humano; reinicio completo.	PASS

M01–M08 corresponden a `TestMinimaxInitializationAndReset`, `TestMinimaxUserAndNPCTurns`, `TestMinimaxBestMoveAndRestoration`, `TestMinimaxWinningLinesAndDepth`, `TestMinimaxGameEndAndNoFurtherMoves`, `TestMinimaxNeverLosesExhaustive`, `TestMinimaxMatchesOriginalCSharpExhaustive` y `TestMinimaxFormIntegration`, respectivamente.

M06 parte del tablero vacío con X; ramifica por cada casilla humana libre, ejecuta la respuesta determinista de Minimax y se detiene al ganar o llenar el tablero. Cuenta caminos terminales, no posiciones distintas. La tasa de derrotas de IA es $0/569 \times 100 = 0\,\%$ *en este conjunto explorado*; no es una medición de partidas de usuarios reales.

M07 enumera estados legales empezando con X y corta cada rama al finalizar. Deduplica 5 478 tableros y compara las 2 097 posiciones no terminales con turno O. Ordena tableros lexicográficamente y concatena nueve caracteres, barra vertical, índice devuelto y salto LF; calcula SHA-256 en UTF-8. La referencia se obtuvo mediante la compilación y ejecución del C# original con un programa auxiliar. La repetición de la ejecución produjo el mismo resultado. Se incorporó la referencia a la prueba Go para permitir la verificación posterior sin depender de un compilador C#.

```
fb6a9200c81c724a24f04ddad024e88659df8a3837e3c2c4c5fb9d6c84d7b31f
```

Esta coincidencia valida decisiones y desempates en ese dominio legal. No mide rendimiento C# frente a Go ni prueba automáticamente el formulario Windows completo.

5.7.4. Integración HTTP, frontend y MediumAI

Para verificar la integración se compiló el servidor Go y se ejecutó en `http://localhost:18084`. Se enviaron peticiones HTTP mediante un script PowerShell para comprobar la creación de partidas, los movimientos, el reinicio y el cambio de modalidad.

Tabla 14: Integración ejecutada y evidencia visual pendiente

ID	Objetivo y procedimiento	Esperado	Obtenido	Estado
I01	GET /api/modes; crear partidas de cada nivel.	Tres modos habilitados y tableros vacíos.	Niveles 1, 2 y 3 disponibles.	PASS
I02	En cada nivel enviar X0 y repetir casilla ocupada.	O4; eventos usuario/IA; ocupada sin nuevos eventos.	Correcto en los tres modos, incluido MediumAI.	PASS
I03	POST /new y PUT /level en cada nivel.	Reinicio y nivel correcto.	Datos reiniciados y nivel seleccionado.	PASS
I04	Minimax: X en 0, 1 y 3; intentar 8 después del final.	Victoria IA y siguiente petición HTTP 409.	AI Wins!, tablero deshabilitado, HTTP 409.	PASS
I05	Compilar Angular en producción con dependencias existentes.	TS, plantillas y estilos compilan.	Build correcto con salida temporal.	PASS
I06	Inspeccionar GameApiService, GameState y GameStore.	Rutas, campos y mensajes compatibles con Go.	Compatibilidad por examen estático.	Revisión
I07	Jugar visualmente en Angular los tres modos.	Selector, animaciones, marcador y resultados visibles.	Validación visual y capturas pendientes.	Pend.

Compilar Angular y probar HTTP no equivale a una prueba de navegador de extremo a extremo.

5.7.5. Espacios para evidencias visuales

Se reservaron las figuras 4–8 para presentar el selector, las partidas y el resultado, y las figuras 9 y 10 para los registros de consola. Los marcadores identifican las capturas pendientes y se sustituyen automáticamente al incorporar cada archivo en su ruta.

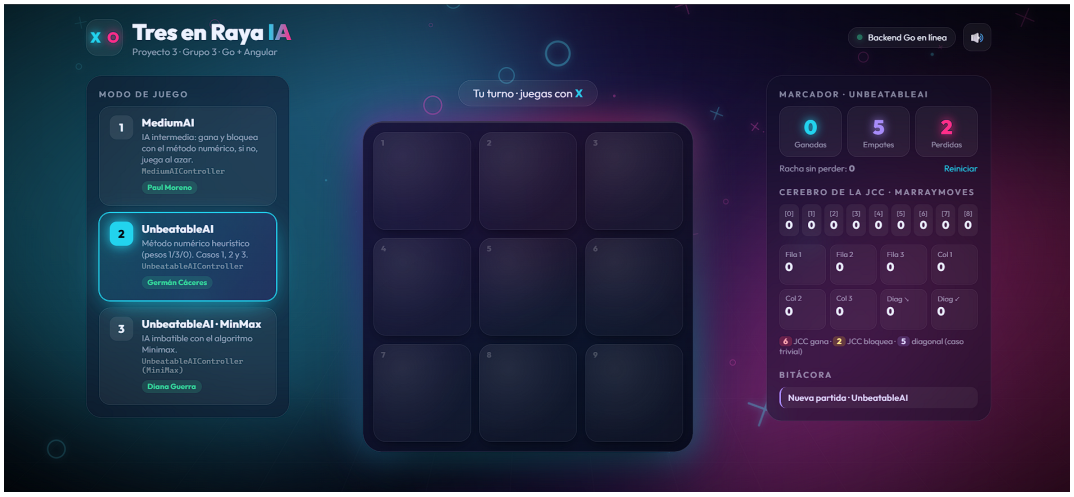


Figura 4: Pantalla principal con MediumAI, UnbeatableAI y Minimax disponibles.

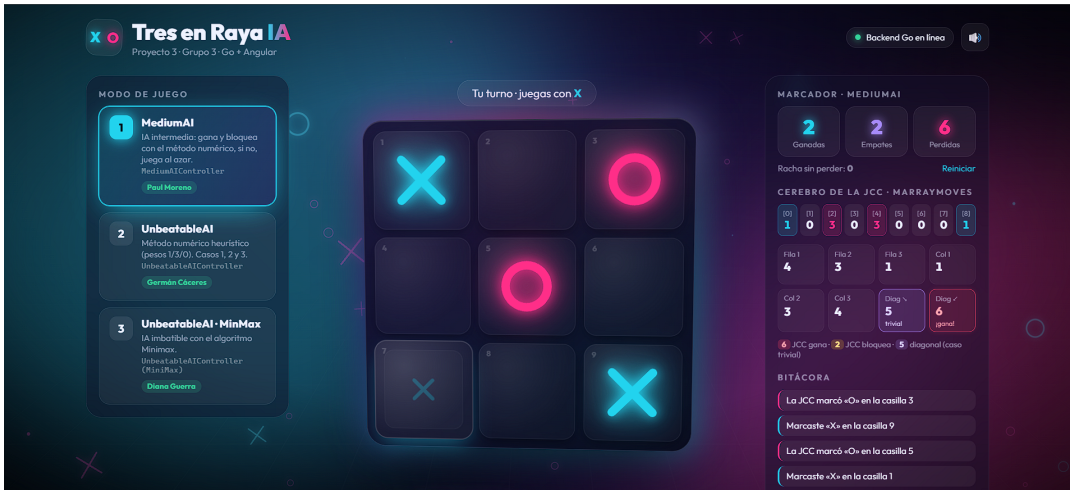


Figura 5: Partida en MediumAI con jugadas humanas y respuesta del backend.

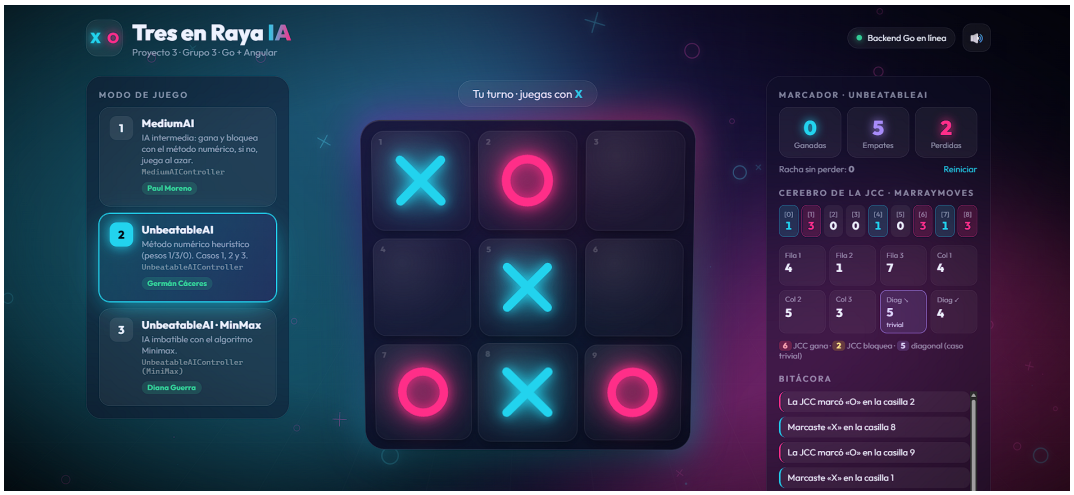


Figura 6: Partida en UnbeatableAI con una regla numérica observable en el inspector.

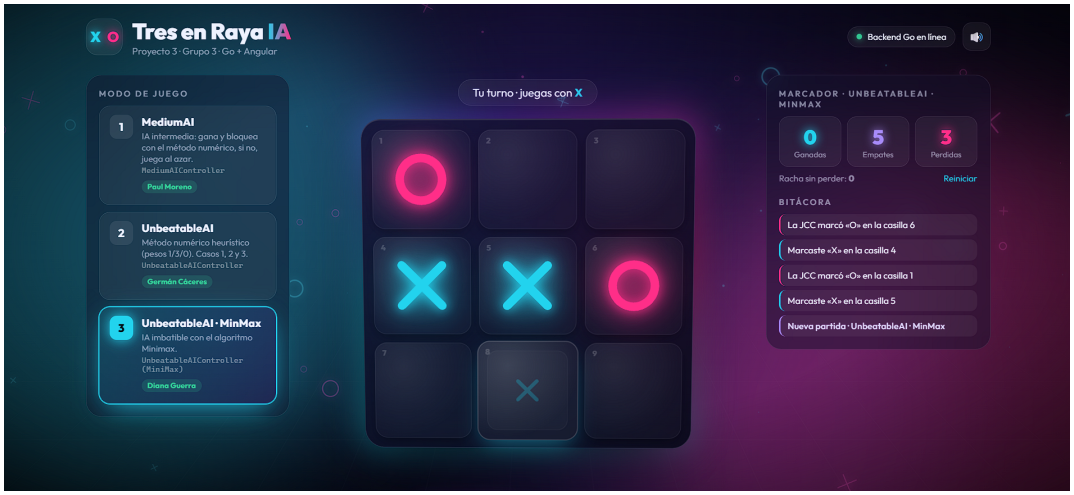


Figura 7: Partida activa en Minimax con tablero y bitácora de movimientos del nivel 3.

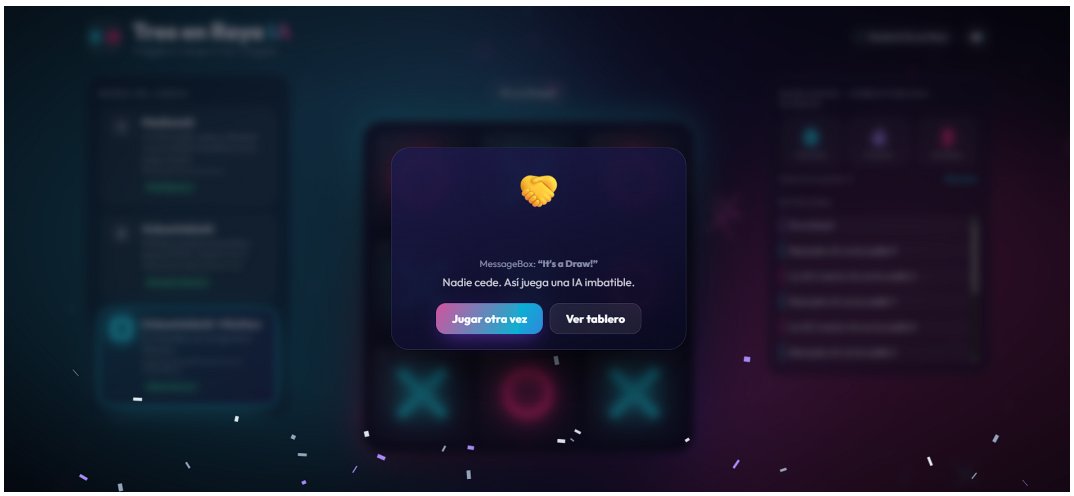


Figura 8: Victoria IA o empate en Minimax, mensaje original y tablero bloqueado.

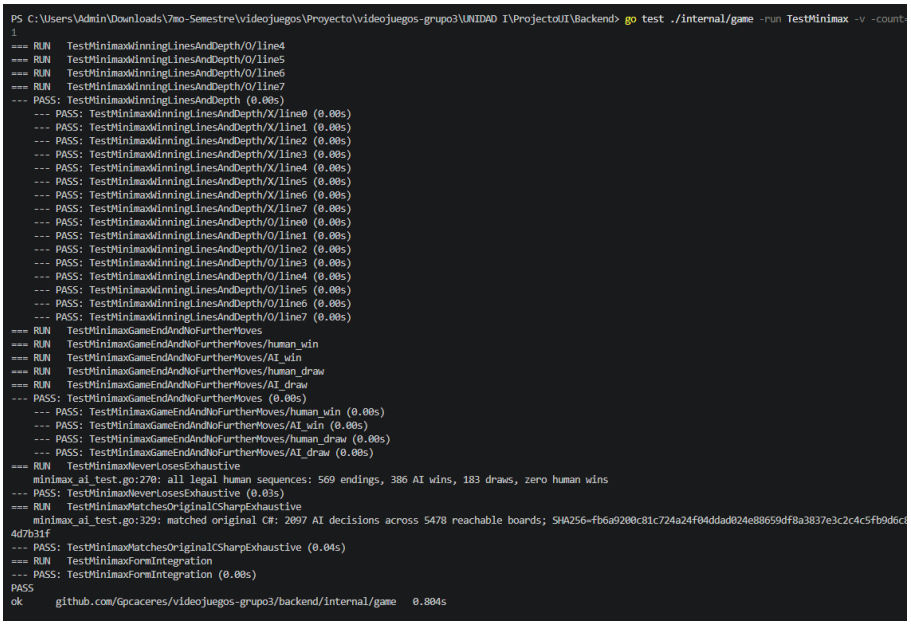


Figura 9: Pruebas Go con conteos de exploración Minimax y comparación con C#.

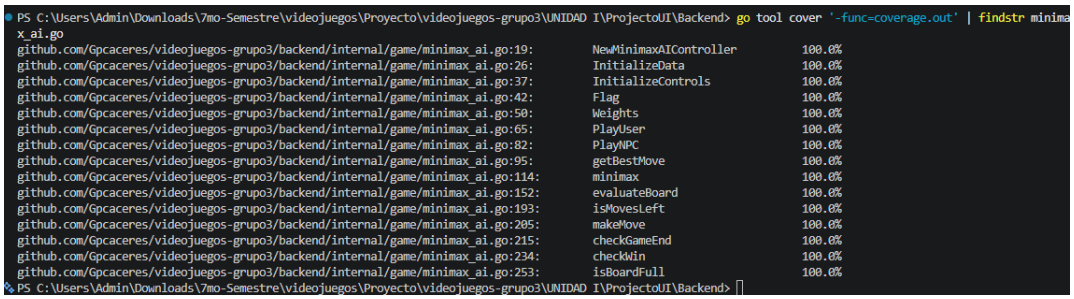


Figura 10: Cobertura del paquete game y de las funciones de minimax_ai.go.

6. Análisis de Resultados: Métricas de Software

6.1. Líneas de código

Se contabilizaron las líneas de código el 2 de noviembre de 2026 mediante un script Node.js, con un criterio uniforme de clasificación de líneas físicas para C#, Go, TypeScript, HTML y SCSS. Una línea sin caracteres no blancos es vacía; si, al retirar comentarios respetando cadenas, queda código, es LOC; si solo queda comentario, es comentario. Una línea con código y comentario cuenta una vez como LOC. Se incluyen llaves, declaraciones e importaciones. Se reconocen comentarios de línea, de bloque y HTML. La última terminación de línea no crea una línea vacía adicional.

Se excluyen dependencias, generados, JSON, binarios, PDF y auxiliares. Backend: archivos Go del módulo, separando `_test.go`. Frontend: TS, HTML y SCSS bajo `Frontend/src`. Se cuentan declaraciones de funciones y métodos; se incluyen constructores explícitos y se excluyen firmas de interfaz, accesores de propiedades C# y funciones anónimas. No se mide complejidad ciclomática.

Tabla 15: Tamaño actualizado por alcance de código fuente

Alcance	Archivos	LOC	Coment.	Blanco	Total
Backend Go: producción	8	1 105	111	119	1 335
Backend Go: pruebas	3	676	30	34	740
Backend Go completo	**11**	**1 781**	**141**	**153**	**2 075**
Frontend Angular: src	24	2 179	38	292	2 509

El backend contiene 78 funciones/métodos de producción y 23 declaraciones en pruebas (24 funciones Test y 7 auxiliares), total 109.

Tabla 16: Controladores por modalidad con criterio uniforme

Archivo	LOC	Coment.	Blanco	Total	Func.
C# MediumAIController	315	83	39	437	13
Go medium_ai.go	240	4	14	258	14
C# UnbeatableAIController (fuerte)	417	83	39	539	13
Go unbeatable_ai.go	223	39	26	288	17
C# UnbeatableAIController (Minimax)	120	3	15	138	4
Go minimax_ai.go	228	17	17	262	15

Se compararon archivos completos con responsabilidades distintas. El controlador Minimax C# contiene 4 métodos de búsqueda, mientras que el archivo Go reúne 15 funciones que incluyen el constructor, la adaptación a Controller y el flujo migrado de frmTicTacToe. Esta diferencia de alcance explica el aumento de LOC. En UnbeatableAI, los valores de 417 y 223 LOC reflejan diferencias de estructura y distribución de responsabilidades. En ambos casos, el tamaño del código se interpreta como una métrica descriptiva, no como una medida de rendimiento.

Tabla 17: Validación de la pareja Minimax C#/Go

Evidencia	Resultado y alcance
C# original ejecutado por arnés	2 097 decisiones en posiciones legales de turno O; referencia SHA-256.
Go: equivalencia	Mismas 2 097 decisiones y SHA-256.
Go: pruebas específicas	8 funciones Test aprobadas; no son pruebas C# independientes.
C#: cobertura	No medida; no se atribuye 100 % al original.
Go: cobertura de minimax_ai.go	100 % de sentencias en sus 15 funciones con la suite instrumentada.

6.2. Otras métricas

Tabla 18: Métricas funcionales por modalidad

Métrica	MediumAI	UnbeatableAI	Minimax
Archivos de controlador / prueba	1 / 1	1 / 1	1 / 1
Funciones del controlador	14	17	15
Funciones Test aprobadas	8 / 8	8 / 8	8 / 8
Finales explorados por sus pruebas	15 982	15 134	569
Derrotas de IA	590	162	0
Porcentaje de derrotas de IA	3,69 %	1,07 %	0 %
Tiempo de suite filtrada (s)	0,819	0,820	0,648
Dependencias externas Go	0	0	0

En UnbeatableAI se recorrieron las opciones humanas con respuestas aleatorias de la IA; en Minimax se exploraron todos los caminos humanos legales frente a una respuesta determinista. Las tasas corresponden a procedimientos de exploración distintos y se interpretan dentro del alcance de cada uno.

Se obtuvo una cobertura de **96,0 % de sentencias de internal/game** con la suite instrumentada y de **100 % de minimax_ai.go** en sus 15 funciones. HeuristicNumericalMethod de MediumAI alcanzó 100 % de cobertura de sentencias. La cobertura de sentencias expresa el código ejecutado durante las pruebas y no implica la evaluación de todos los caminos posibles.

6.2.1. Entorno y tiempos de pruebas

Se ejecutaron las pruebas en Windows NT 10.0.26200, arquitectura amd64, con un procesador AMD Ryzen 5 7520U, Go 1.27.3, Node.js 24.19.0 y las dependencias Angular 22.2.x del proyecto. Los tiempos corresponden a una ejecución local. Se utilizó `-count=1` para evitar reutilizar resultados y se ubicó la caché de compilación Go en una carpeta temporal. `go test -count=1 -v ./...` reportó 0,913 s para game; la ejecución con cobertura reportó 0,821 s.

Se ejecutó también `go test -json -count=1 -run` con el patrón `^TestMinimax` o el conjunto de nombres U01–U08. El campo Elapsed del registro final del paquete fue 0,648 s y 0,820 s, respectivamente; no es tiempo puro del algoritmo. M06 tardó 0,02 s, M07 0,03 s y U08 0,02 s con la resolución informada por Go. Los conteos aleatorios de U08 pueden diferir de la ejecución general.

6.2.2. Tiempo de respuesta HTTP

Se compiló `./cmd/server` con `go build -o <temporal>/api.exe` y se inició con `PORT=18084`. Para cada nivel se creó una partida vacía y se midió `POST /api/games/{id}/moves` con `index=0`, verificando O en 4. Se descartó una petición de calentamiento por nivel y se tomaron 30 muestras con `System.Diagnostics.Stopwatch`

alrededor de `Invoke-RestMethod`. La creación quedó fuera del intervalo. Se ordenaron las 30 muestras: mediana = promedio de posiciones 15 y 16; P95 = posición 29 ($\lceil 0{,}95 \times 30 \rceil$).

Tabla 19: Latencia HTTP local de la primera jugada, en milisegundos

Modo	Muestras	Mínimo	Mediana	P95	Máximo
MediumAI	30	1,416	1,683	2,692	8,185
UnbeatableAI	30	1,252	1,449	2,159	2,163
Minimax	30	3,232	3,464	5,587	5,819

Los tiempos incluyen cliente PowerShell, HTTP y deserialización JSON; no son exclusivos del algoritmo ni incluyen las pausas de animación Angular. Todas las muestras estuvieron bajo 50 ms, con cobertura parcial de RNF02. El ensayo se limitó a la primera jugada en un entorno local y sin carga concurrente; por tanto, los valores describen ese escenario de uso.

6.2.3. Dependencias y compilación frontend

`go list -m all` devolvió solo el módulo del proyecto: cero dependencias externas del backend. Con las dependencias Angular ya presentes, `ng build -configuration production -output-path <temporal> -progress=false` terminó correctamente: paquete inicial 216,30 kB, transferencia estimada 61,95 kB y 5,505 s informados por el compilador. La transferencia es una estimación del build. La validación de Angular comprendió la compilación y la comprobación del contrato HTTP; las pruebas de componentes quedan pendientes.

6.3. Comportamiento de la variante UnbeatableAI

Se identificaron los siguientes comportamientos de la estrategia numérica de UnbeatableAI y de su migración a Go.

1. **Derrotas observadas.** Se registraron **162 derrotas de la JCC en 15 134 finales (1,07 %)** durante el recorrido con muestreo. La tasa corresponde a esa ejecución y depende de las respuestas aleatorias. Cuando ninguna regla aplica, la lógica cae en `GenerateRandomNPCMove` y a veces deja al usuario un *tenedor* (dos amenazas a la vez). Ejemplo: X en 0, O en 4, X en 7, O en 2 (al azar), X en 6. La JCC solo puede tapar una de las amenazas (0–3–6 y 6–7–8), y el usuario gana.

2. **La regla de suma 5 excluye el lado 3.** En `PlayNPC`, la condición original del lado aleatorio compara `SIDE_INDICES[0]` dos veces (`index == SIDE_INDICES[0] || index == SIDE_INDICES[0] || ...`), así que el lado 3 queda fuera de esa selección restringida. Go pasa la misma lista a `randomFreeAmong`; la prueba `TestSideRuleNeverPicksThree` verifica esa regla. El lado 3 permanece disponible para los movimientos aleatorios que no aplican esta restricción.

3. **Posible bloqueo.** Los `do/while` aleatorios no tienen salida si no queda casilla válida del grupo. En los 15 134 finales citados se registraron **0 casos**. Para evitar el bloqueo de una petición, se incorporó la comprobación de disponibilidad descrita en RNF07.

4. **Regla del centro sin efecto.** A partir de la segunda jugada, `PlayNPC` revisa si el centro está libre; nunca lo está a esa altura, porque la primera jugada de alguno de los dos ya lo ocupó.

Con **Minimax** se obtuvieron cero derrotas en los 569 finales explorados frente a todos los movimientos humanos legales. Este resultado corresponde a una estrategia determinista y se distingue del muestreo de UnbeatableAI.

7. Conclusiones y Recomendaciones

7.1. Conclusiones

- Se integraron las tres modalidades de IA en una arquitectura Go + Angular. La separación entre lógica y presentación permitió sustituir Windows Forms por una interfaz web y mantener las decisiones de juego en el servidor.
- Se implementaron estrategias con distintos criterios de selección. MediumAI utiliza centro, victoria, bloqueo y azar; UnbeatableAI incorpora reglas de diagonales, esquinas y lados. En esta última se registraron 162 derrotas de IA en 15 134 finales (1,07 %), con variación asociada a las respuestas aleatorias.
- La migración de Minimax de C# a Go permitió conservar su comportamiento funcional, incluidas las puntuaciones, el orden de evaluación, la restauración del tablero y los desempates. Coincidieron las 2 097 decisiones comparadas; en 569 finales se obtuvieron 386 victorias de IA, 183 empates y ninguna derrota.
- Se aprobaron 23 funciones Test y se ejecutó go vet sin diagnósticos. La cobertura alcanzó 96,0 % en game y 100 % de sentencias en Minimax. Estos resultados complementan la comparación funcional y la exploración de partidas, con el alcance propio de cada técnica de validación.
- Se verificaron la compilación Angular y la comunicación HTTP con Go. Las pruebas específicas de MediumAI verificaron inicialización, turnos, centro, reglas heurísticas, azar y estados terminales. Queda pendiente la comprobación visual de los tres modos en navegador.

7.2. Recomendaciones

- Completar la validación visual I07 e incorporar las siete capturas preparadas para documentar la interfaz y los resultados.
- Ampliar las pruebas de navegador, manteniendo resultados y métricas diferenciados por modalidad.
- Mantener la referencia SHA-256 y las pruebas de Minimax para detectar cambios de comportamiento durante futuras modificaciones de infraestructura.
- Repetir los ensayos de latencia en el entorno de despliegue y con carga concurrente para evaluar el objetivo local de 50 ms en otros escenarios.

Referencias

[1] C. Villacís Silva, *Informe de laboratorio Desarrollo de Videojuegos*, Unidad 1, indicaciones del Proyecto 3. Archivo local: `Informe-Lab-Desarrollo-Videojuegos.docx`.

[2] Material del docente, *Tic-Tac-Toe-Version-MiniMax*: `UnbeatableAIController.cs`, `frmTicTacToe.cs` y `minmax-ia-explicacion.docx`, código y explicación originales incluidos en `UNIDAD I/Material`.

[3] IEEE, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998.

[4] C. Villacís, W. Fuertes, A. Bustamante et al., *Optimización del Juego del Tres en Raya con Niveles de Dificultad, Utilizando Técnicas Heurísticas de Inteligencia Artificial*, Universidad de las Fuerzas Armadas – ESPE.

[5] The Go Authors, *The Go Programming Language Documentation*, `https://go.dev/doc/`.

[6] Google, *Angular Documentation*, `https://angular.dev`.

[7] K. Schwaber y J. Sutherland, *La Guía de Scrum*, 2020.

[8] K. Beck, *Extreme Programming Explained: Embrace Change*, Addison-Wesley, 2004.